

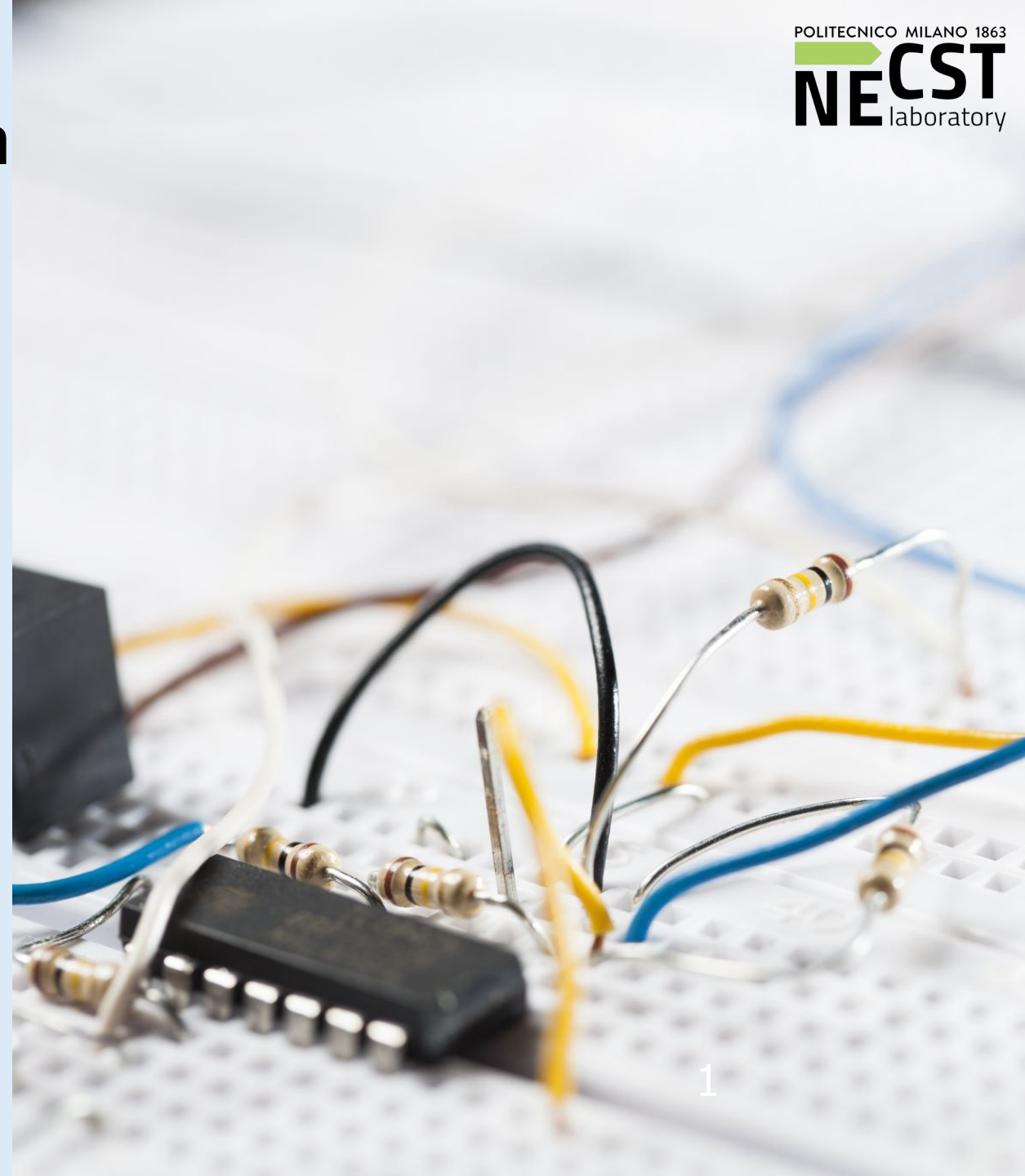
Creativity, Science and **Innovation**

Ideate, invent, innovate:
from ideas to prototypes

Additional material

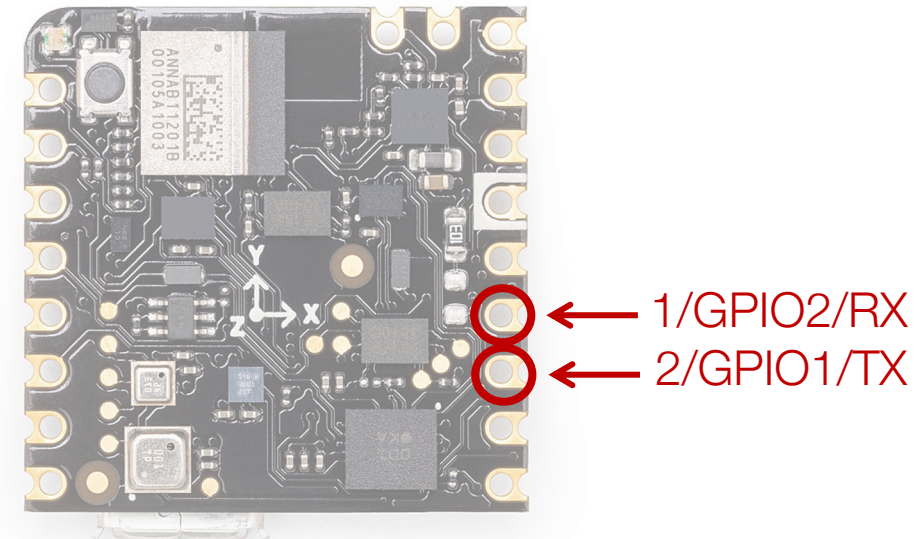
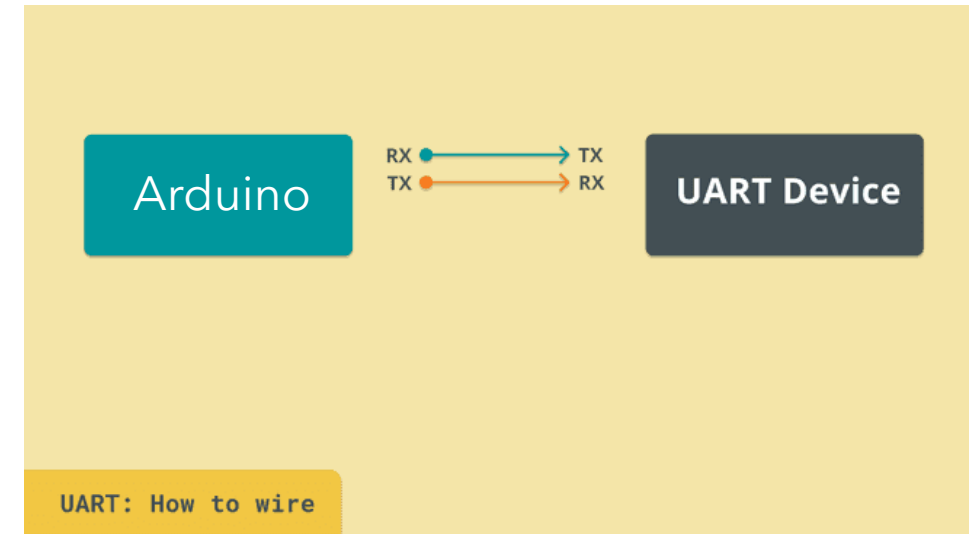
Arduino Nicla SENSE ME

Susanna Bardini
susanna.bardini@polimi.it



Universal asynchronous receiver-transmitter (UART)

- Communication with the PC can be established via the **USB port**. The simplest way to view serial data on the PC is with the Serial Monitor in the Arduino IDE.
- **Asynchronous**: there is **no** need for a **shared clock** signal between the transmitter and receiver. Based on same baudrate.
- All Arduino boards have at least one **serial port**.
- Pins: **Rx (Receive)** and **Tx (Transmit)**, GPIO2 and GPIO1 for this specific board respectively. When using the serial connection, these pins cannot be used for other purposes.



Universal asynchronous receiver-transmitter (UART)

The devices must be configured to have same baud rate, parity bits, and other communication settings.

The void `setup()` function is used to initialize the serial communication:

- `Serial.begin(baudRate);`
- `baudRate`: is the rate at which data is transmitted over a communication channel, measured in bits per second (bps); it must match the PC's rate.

In void `loop()` we list all the instructions:

To **write** to the serial output (to the PC):

- `Serial.print(val)`: writes data using ASCII representation
- `Serial.println(val)`: as above, but adds terminator at the end (`\n`)
- `Serial.write(val)`: writes the input data as a stream of bytes

Universal asynchronous receiver-transmitter (UART)

In void `loop()`:

To **read** from the serial port (from the PC):

- `Serial.read()`: returns the first available character (in byte format) present in the buffer, otherwise -1 is there are no data available
- `Serial.parseInt()`: returns the conversion to integers of the bytes read, stopping at the first byte that does not represent a number. If no data is available returns 0
- `Serial.parseFloat()`: returns the conversion to floating point number of the bytes read, stopping at the first byte that does not represent a number.

To check if there is data available to read (thus avoiding waiting to read, or to read «empty») you can use the function `Serial.available()`, which returns the number of bytes available to read

Universal asynchronous receiver-transmitter (UART)

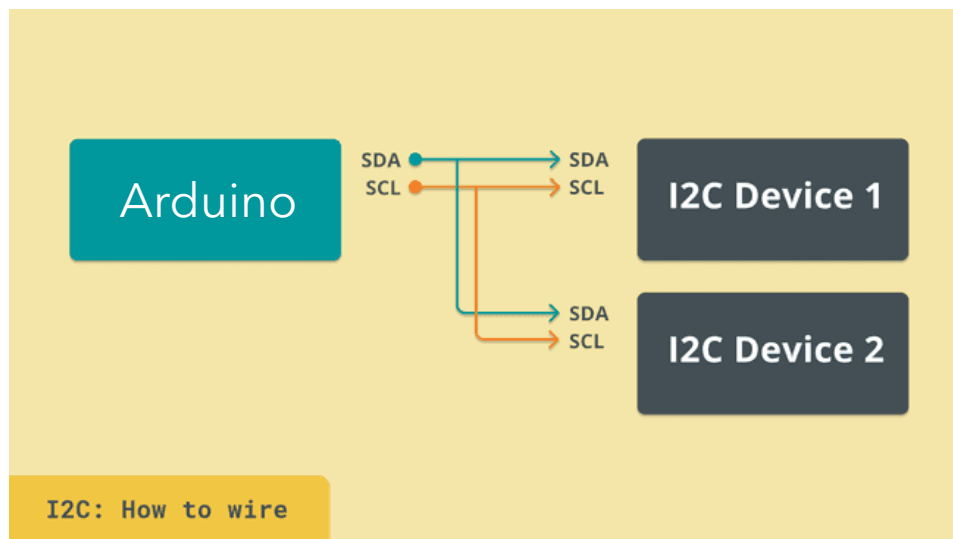
```
int ledPin = 13; // Pin where the LED is connected
```

```
void setup() {  
    Serial.begin(9600); // Initialize serial communication at 9600 baud rate  
  
    pinMode(ledPin, OUTPUT); // Set the LED pin as output  
    Serial.println("Enter '1' to turn LED ON, '0' to turn LED OFF");  
}
```

Universal asynchronous receiver-transmitter (UART)

```
void loop() {  
  if (Serial.available() > 0) { // Check if data is available to read  
    char incomingChar = Serial.read(); // Read the incoming byte as a character  
  
    if (incomingChar == '1') {  
      digitalWrite(ledPin, HIGH); // Turn LED on  
      Serial.println("LED is ON");  
    }  
  
    else if (incomingChar == '0') {  
      digitalWrite(ledPin, LOW);  
      Serial.println("LED is OFF"); // Turn LED off  
    }  
  }  
}
```

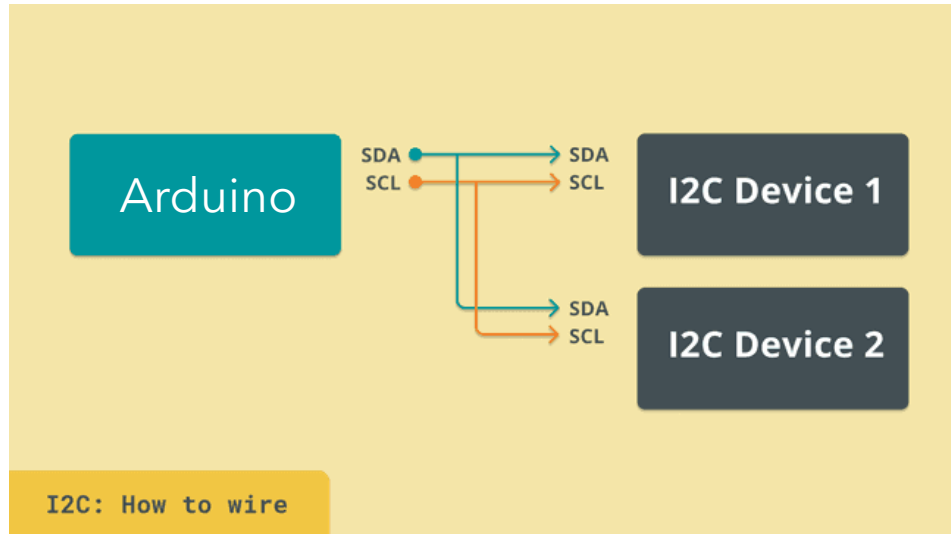
Inter-Integrated Circuit (I2C)



I2C is a serial protocol that allows you to connect many devices (up to 128) on the same two wires: Serial Data and Serial Clock.

Each component that uses I2C has a unique address of 7 or 10 bit).

Inter-Integrated Circuit (I2C)



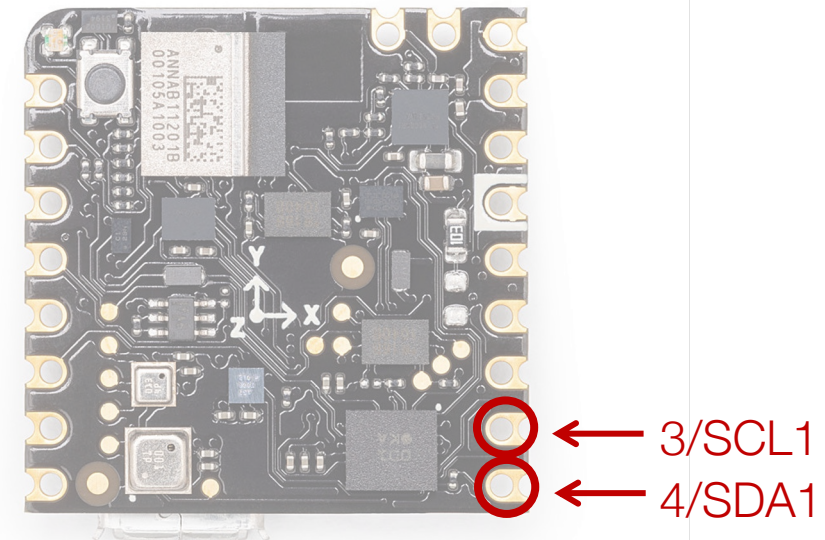
I2C is a serial protocol that allows you to connect many devices (up to 128) on the same two wires: Serial Data and Serial Clock.

Each component that uses I2C has a unique address of 7 or 10 bit.

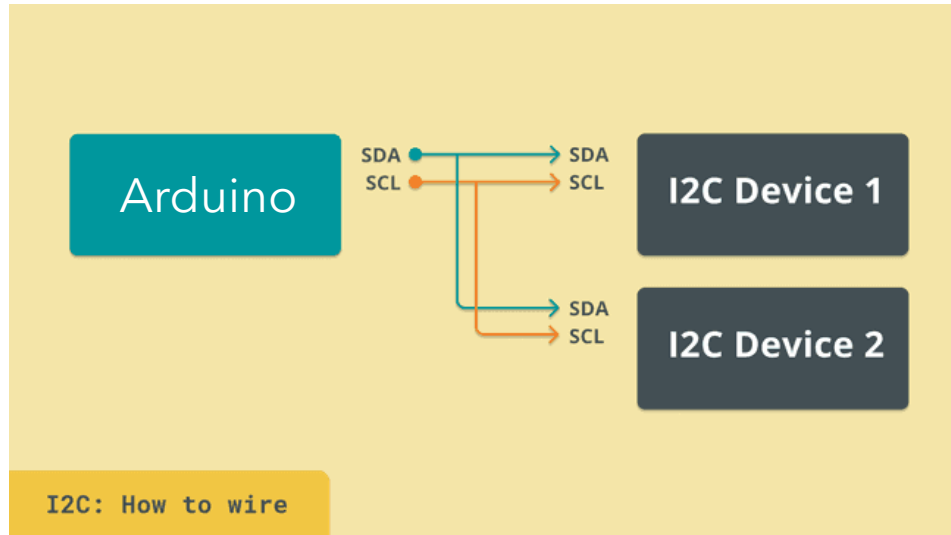
When the master communicates to a device:

- Start condition
- Device unique address (0X0C)
- Write/read flag (0/1)
- Data
- Stop condition

Often, data is accessed via internal registers inside the device, which have their own sub-addresses.



Inter-Integrated Circuit (I2C)



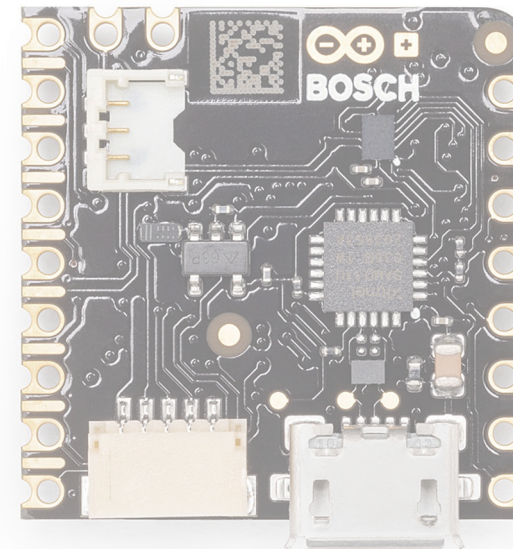
I2C is a serial protocol that allows you to connect many devices (up to 128) on the same two wires: Serial Data and Serial Clock.

Each component that uses I2C has a unique address of 7 or 10 bit).

When the master has to communicate to a device:

- Start condition
- Device unique address (0X0C)
- Write/read flag (0/1)
- Data
- Stop condition

Often, data is accessed via internal registers inside the device, which have their own sub-addresses.



Inter-Integrated Circuit (I2C)

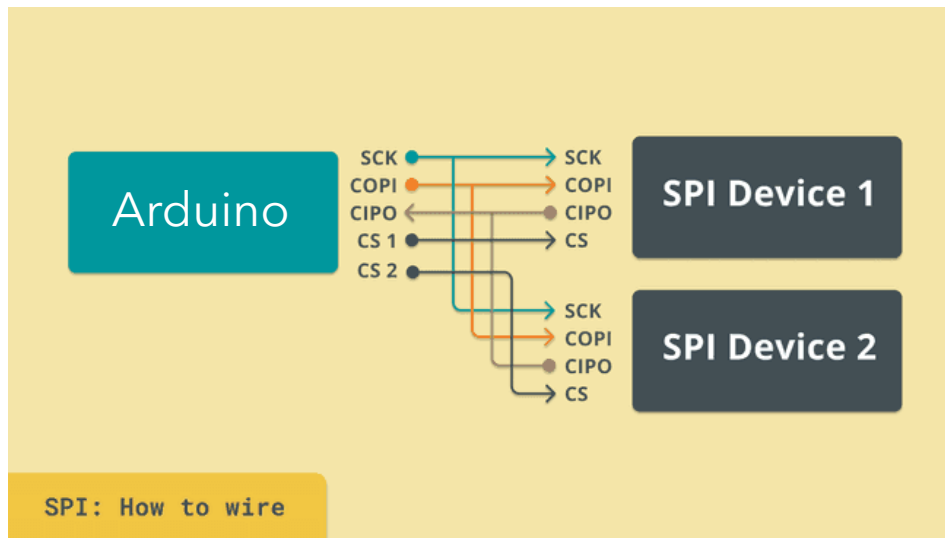
```
#include <Wire.h>
```

```
const byte deviceAddress = 0x35; // Target device's I2C address to which we want to communicate  
const byte instruction = 0x00; // Instruction byte to be sent  
const byte value = 0xFA; // Value to be sent
```

```
void setup() {  
    Wire.begin(); // Initialize the SPI communication  
}
```

```
void loop(){  
    Wire.beginTransmission(deviceAddress); // Begin transmission to the target device  
    Wire.write(instruction); // Send the instruction byte  
    Wire.write(value); // Send the value  
    Wire.endTransmission(); // End transmission  
}
```

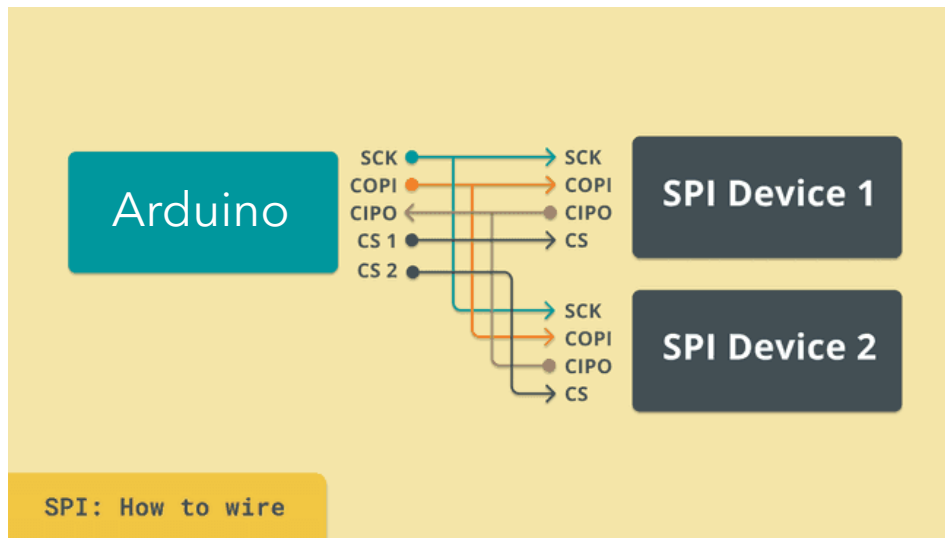
Serial Peripheral Interface (SPI)



SPI is a 4-wire serial protocol and is also used for communication between systems.

It is in general twice as fast as I2C, but with more complexity in adding more devices to it.

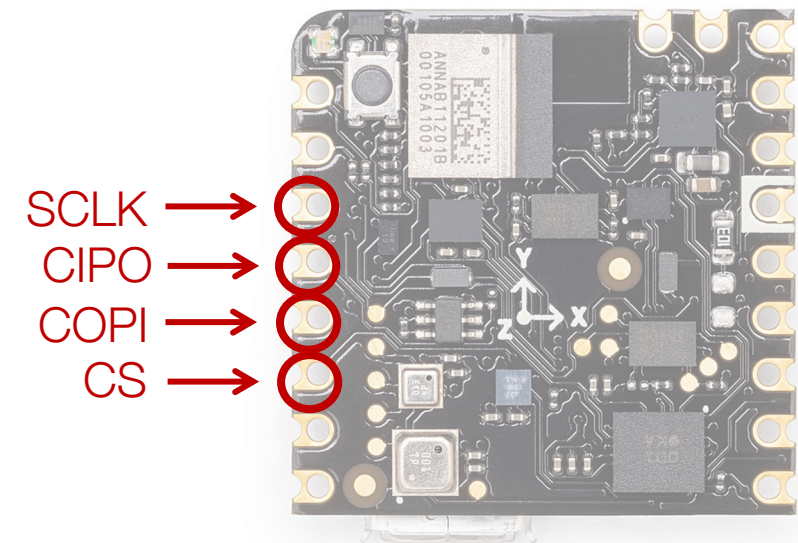
Serial Peripheral Interface (SPI)



SPI is a 4-wire serial protocol and is also used for communication between systems.

It is in general twice as fast as I2C, but with more complexity in adding more devices to it.

- MOSI (Master Out Slave In): master sends data
- MISO (Master In Slave Out): slave sends data
- SCLK (Serial Clock): generated by the master
- SS/CS (Slave Select / Chip Select): line dedicated to slave selection. We need a CS dedicated to each connected slave device



Serial Peripheral Interface (SPI)

```
#include <SPI.h>
```

```
const int chipSelectPin = 10; // Define a pin CS. It allows to activate the external device
```

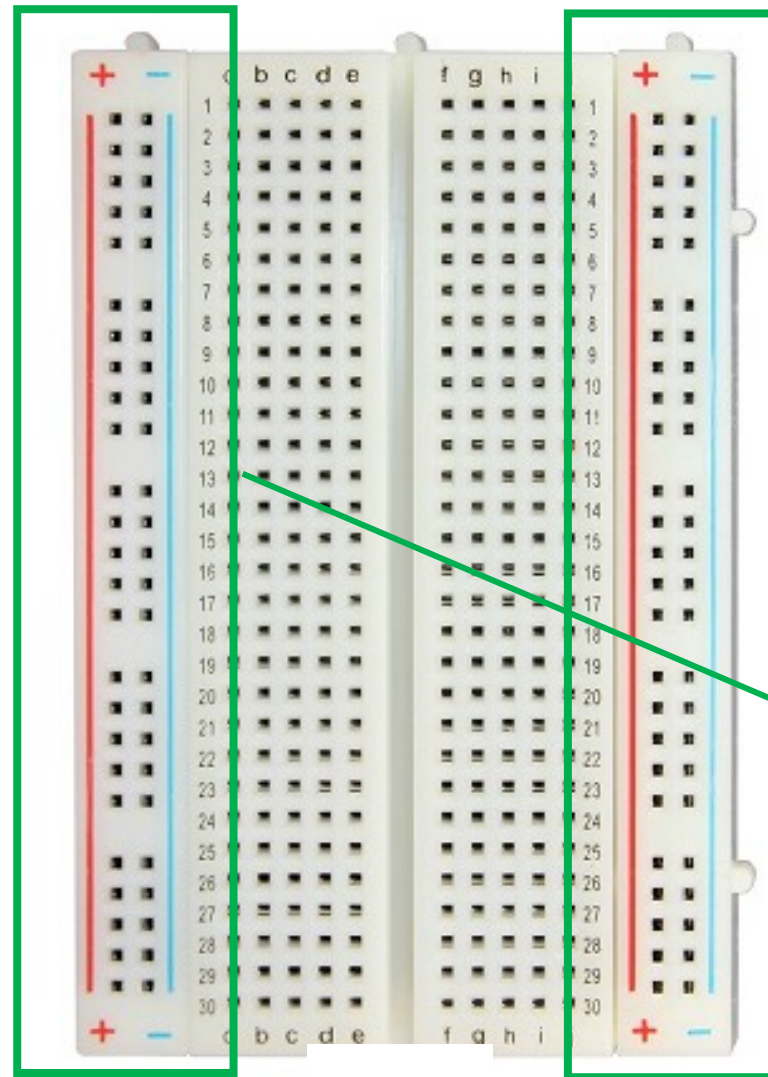
```
void setup() {  
    pinMode(chipSelectPin, OUTPUT); // Set the chip select pin as output  
    digitalWrite(chipSelectPin, HIGH); // Pull the CS pin HIGH to unselect the device  
    SPI.begin(); // Initialize the SPI communication  
}
```

```
void loop(){  
    byte address = 0x35; // Address or command  
    byte value = 0xFA; // Value to send  
    digitalWrite(chipSelectPin, LOW); // Pull the CS pin LOW to select the connected device  
    SPI.transfer(address); // Send the address  
    SPI.transfer(value); // Send the value  
    digitalWrite(chipSelectPin, HIGH); // Pull the CS pin HIGH to unselect the device  
}
```

How to read data from sensors?

Measurements with external sensors

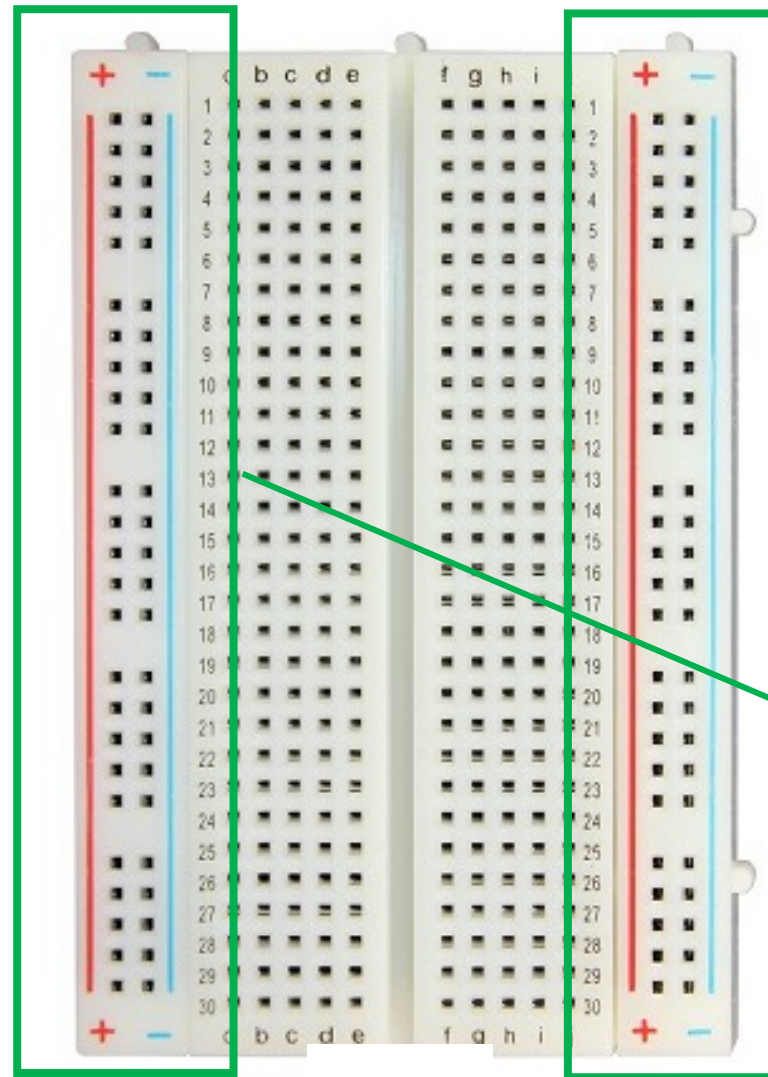
A **breadboard** is a tool used for prototyping and testing electronic circuits without soldering



Power rails

Measurements with external sensors

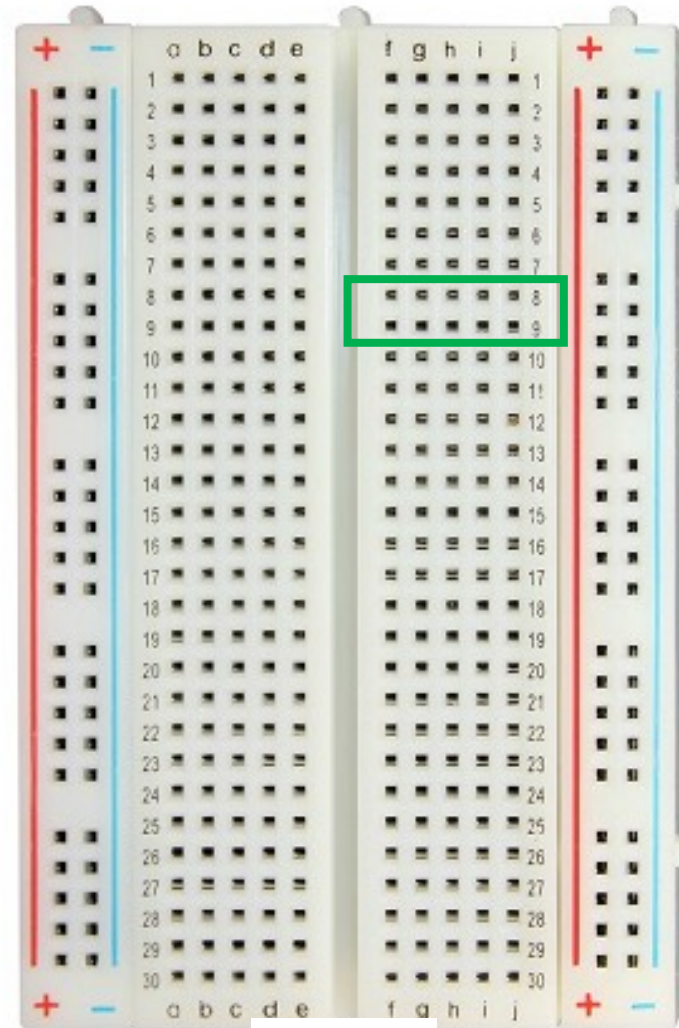
A **breadboard** is a tool used for prototyping and testing electronic circuits without soldering



Power rails

Measurements with external sensors

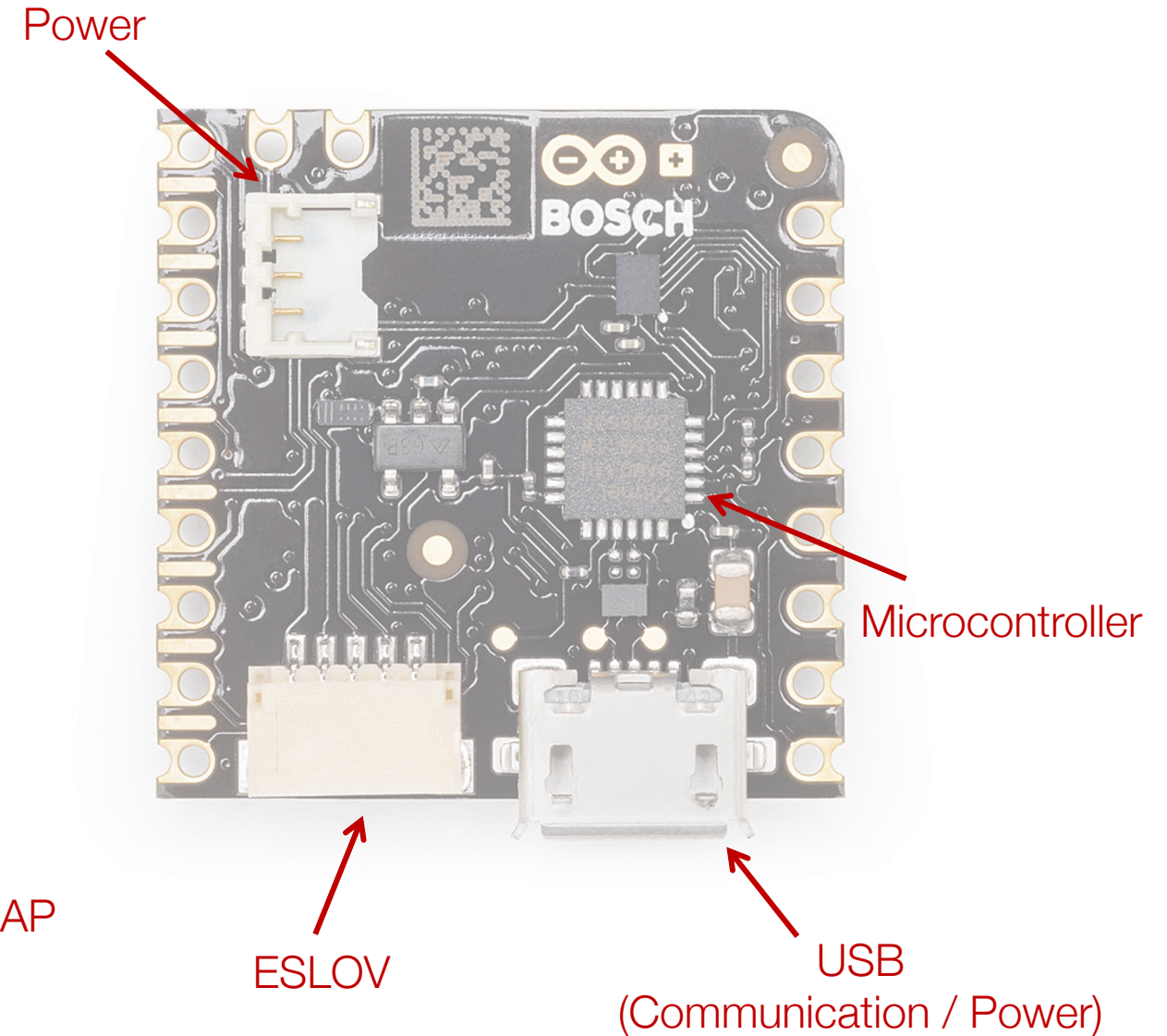
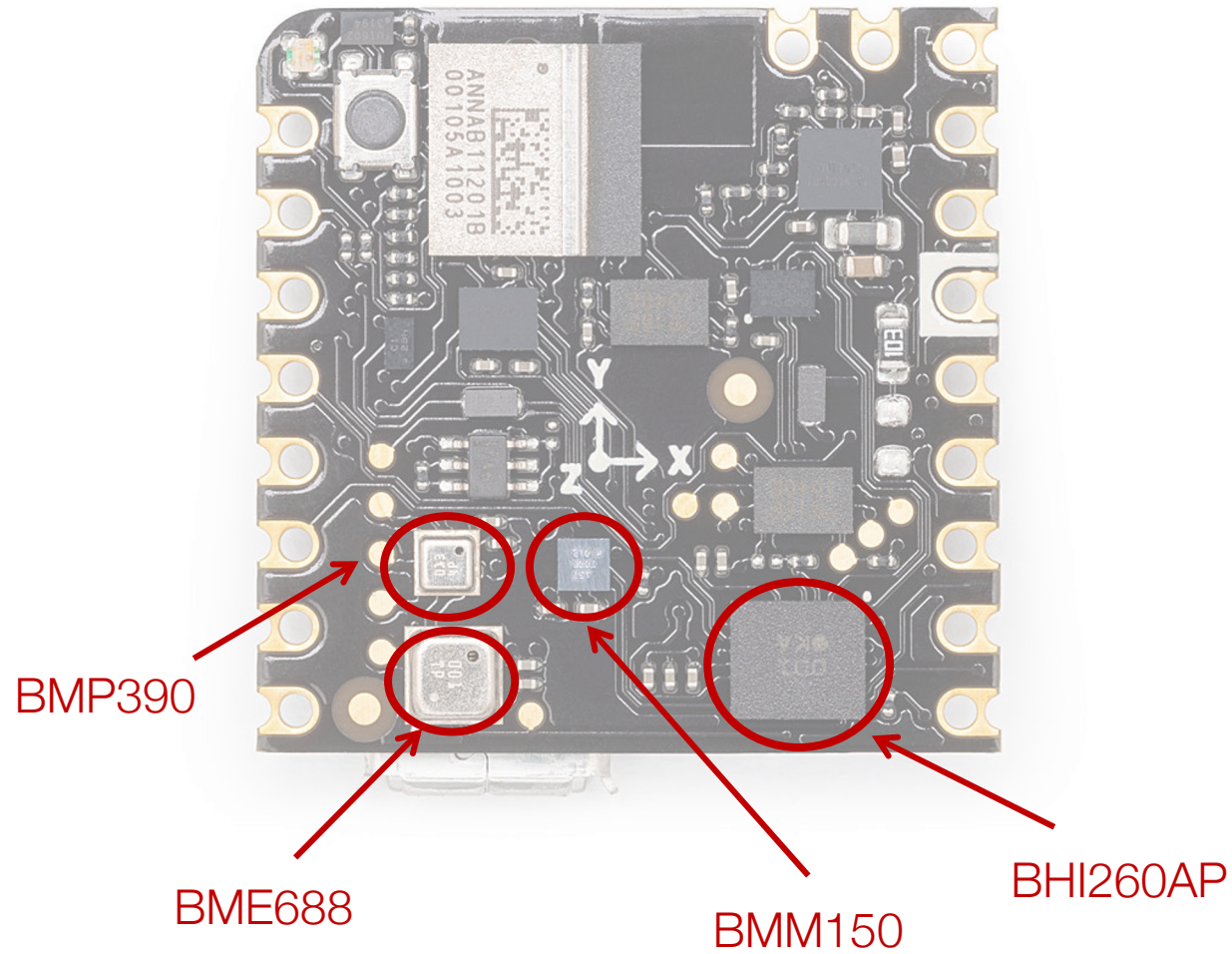
Terminal Strips: The middle area of the breadboard is divided into rows and columns.



The rows are typically marked by numbers (e.g., 1, 2, 3, etc.). Each row contains a set of connected terminals that are electrically connected in parallel..

The columns are typically marked by letters (e.g., A, B, C, D, E). Components placed in the same column within the same row are not electrically connected.

Nicla Sense ME



Embedded sensors

There are three ways to read from the on-board sensors:

1. Read the sensors directly from Nicla Sense ME in standalone mode.
2. Read sensor values through I2C by connecting an ESLOV cable.
3. Read sensor values through Bluetooth Low Energy.

Embedded sensors

There are three ways to read from the on-board sensors:

1. Read the sensors directly from Nicla Sense ME in **standalone mode**
2. Read sensor values through I2C by connecting an ESLOV cable.
3. Read sensor values through Bluetooth Low Energy



In the **standalone mode**, the sensors can be accessed through **sensor objects**. The sensor data is then read by Nicla Sense ME's on-board microcontroller.

Libraries that we need to install to read from the sensors in any of these mode:

- **Arduino_BHY2**

To install libraries: from the Arduino IDE, select Tools->Manage Libraries

Embedded sensors

To use the sensors in your sketches, you need to know the sensors ID, which can be found in:

<https://docs.arduino.cc/tutorials/nicla-sense-me/cheat-sheet/>

<https://github.com/arduino-libraries/Arduino BHY2/blob/main/src/sensors/SensorID.h>

Sensors can be divided in classes:

- **Sensor**: all the other sensors which have a single value to be read (value property)
- **SensorOrientation**: sensors with the Euler format, allows to read pitch, roll, heading property
- **SensorXYZ**: sensors with XYZ format
- **SensorQuaternion**: sensor with the quaternion format (rotation, game rotation and geomagnetic rotation vector)
- **SensorActivity**: sensor with the activity format (getActivity function)
- **SensorBSEC**: Bosch Sensortec Environmental Cluster (IAQ data)

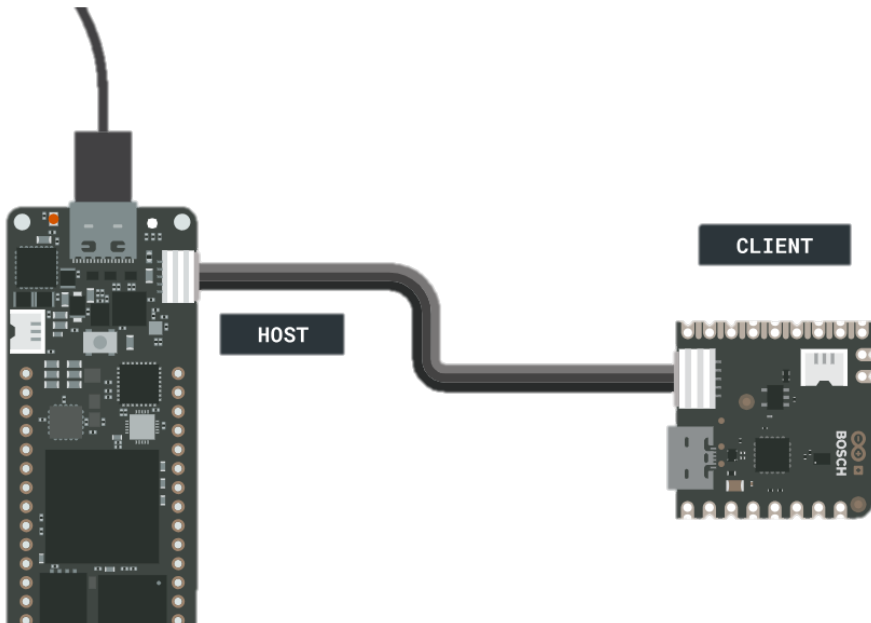
Embedded sensors

There are three ways to read from the on-board sensors:

1. Read the sensors directly from Nicla Sense ME in standalone mode.
2. Read sensor values through I2C by connecting an **ESLOV cable**
3. Read sensor values through Bluetooth Low Energy.

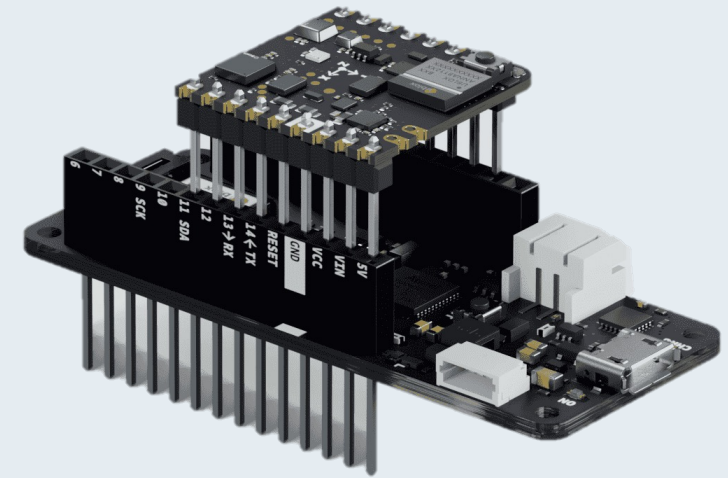


The MKR WiFi 1010 (host) communicates with Nicla Sense ME via the **BHY2Host** library.



Arduino Nicla Sense ME as a MKR Shield

1. Solder two rows of headers: position the long side of the headers on the **battery connectors side**.
2. To connect the boards, plug the Nicla Sense ME on top of the MKR board, ensuring the **LED and Reset button** on the Nicla are facing upwards.



Embedded sensors

There are three ways to read from the on-board sensors:

1. Read the sensors directly from Nicla Sense ME in standalone mode.
2. Read sensor values through I2C by connecting an ESLOV cable.
3. Read sensor values through **Bluetooth Low Energy**



Libraries that we need to install to read from the sensors in any of these mode:

- **Arduino_BHY2**
- **ArduinoBLE**

To install libraries: from the Arduino IDE, select Tools->Manage Libraries

IoT Integration with BLE

The Nicla Sense ME includes a BLE module for communication with nearby devices, such as smartphones, tablets, or IoT gateways.

1. Sensor Data Transmission:

The board collects sensor data and sends it via BLE to a host device (e.g., smartphone).

2. Command Reception:

The Nicla can receive commands to configure sensors, initiate processes, or update parameters.

Advantages of BLE for IoT:

Low Power Consumption:
Ideal for battery-powered applications.

Ease of Implementation:
Many platforms offer built-in BLE support.

Adequate Range:
Suitable for most indoor applications.